JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class LogRecord

java.lang.Object
    java.util.logging.LogRecord

All Implemented Interfaces:

Serializable

```
public class LogRecord
extends Object
implements Serializable
```

LogRecord objects are used to pass logging requests between the logging framework and individual log Handlers.

When a LogRecord is passed into the logging framework it logically belongs to the framework and should no longer be used or updated by the client application.

Note that if the client application has not specified an explicit source method name and source class name, then the LogRecord class will infer them automatically when they are first accessed (due to a call on getSourceMethodName or getSourceClassName) by analyzing the call stack. Therefore, if a logging Handler wants to pass off a LogRecord to another thread, or to transmit it over RMI, and if it wishes to subsequently obtain method name or class name information it should call one of getSourceClassName or getSourceMethodName to force the values to be filled in.

Serialization notes:

- The LogRecord class is serializable.
- Because objects in the parameters array may not be serializable, during serialization all objects in the parameters array are written as the corresponding Strings (using Object.toString).

- The ResourceBundle is not transmitted as part of the serialized form, but the resource bundle name is, and the recipient object's readObject method will attempt to locate a suitable resource bundle.

Since:

1.4

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`LogRecord(Level level, String msg)`	Construct a LogRecord with the given level and message values.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`Level`	`getLevel()`	Get the logging message level, for example Level.SEVERE.
`String`	`getLoggerName()`	Get the source Logger's name.
`String`	`getMessage()`	Get the "raw" log message, before localization or formatting.
`long`	`getMillis()`	Get event time in milliseconds since 1970.
`Object[]`	`getParameters()`	Get the parameters to

		the log message.
ResourceBundle	getResourceBundle()	Get the localization resource bundle
String	getResourceBundleName()	Get the localization resource bundle name
long	getSequenceNumber()	Get the sequence number.
String	getSourceClassName()	Get the name of the class that (allegedly) issued the logging request.
String	getSourceMethodName()	Get the name of the method that (allegedly) issued the logging request.
int	getThreadID()	Get an identifier for the thread where the message originated.
Throwable	getThrown()	Get any throwable associated with the log record.
void	setLevel(Level level)	Set the logging message level,

		for example Level.SEVERE.
void	**setLoggerName**(**String** name)	Set the source Logger's name.
void	**setMessage**(**String** message)	Set the "raw" log message, before localization or formatting.
void	**setMillis**(long millis)	Set event time.
void	**setParameters**(**Object**[] parameters)	Set the parameters to the log message.
void	**setResourceBundle**(**ResourceBundle** bundle)	Set the localization resource bundle.
void	**setResourceBundleName**(**String** name)	Set the localization resource bundle name.
void	**setSequenceNumber**(long seq)	Set the sequence number.
void	**setSourceClassName**(**String** sourceClassName)	Set the name of the class that (allegedly) issued the logging request.
void	**setSourceMethodName**(**String** sourceMethodName)	Set the name of the method that (allegedly) issued the logging request.

void	**setThreadID**(int threadID)	Set an identifier for the thread where the message originated.
void	**setThrown**(**Throwable** thrown)	Set a throwable associated with the log event.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

LogRecord

```
public LogRecord(Level level,
                 String msg)
```

Construct a LogRecord with the given level and message values.

The sequence property will be initialized with a new unique value. These sequence values are allocated in increasing order within a VM.

The millis property will be initialized to the current time.

The thread ID property will be initialized with a unique ID for the current thread.

All other properties will be initialized to "null".

Parameters:

`level - a logging level value`

`msg - the raw non-localized logging message (may be null)`

Method Detail

getLoggerName

```
public String getLoggerName()
```

Get the source Logger's name.

Returns:

source logger name (may be null)

setLoggerName

```
public void setLoggerName(String name)
```

Set the source Logger's name.

Parameters:

name - the source logger name (may be null)

getResourceBundle

```
public ResourceBundle getResourceBundle()
```

Get the localization resource bundle

This is the ResourceBundle that should be used to localize the message string before formatting it. The result may be null if the message is not localizable, or if no suitable ResourceBundle is available.

Returns:

the localization resource bundle

setResourceBundle

```
public void setResourceBundle(ResourceBundle bundle)
```

Set the localization resource bundle.

Parameters:

bundle - localization bundle (may be null)

getResourceBundleName

```
public String getResourceBundleName()
```

Get the localization resource bundle name

This is the name for the ResourceBundle that should be used to localize the message string before formatting it. The result may be null if the message is not localizable.

Returns:

`the localization resource bundle name`

setResourceBundleName

```
public void setResourceBundleName(String name)
```

Set the localization resource bundle name.

Parameters:

`name - localization bundle name (may be null)`

getLevel

```
public Level getLevel()
```

Get the logging message level, for example Level.SEVERE.

Returns:

`the logging message level`

setLevel

```
public void setLevel(Level level)
```

Set the logging message level, for example Level.SEVERE.

Parameters:

`level - the logging message level`

getSequenceNumber

```
public long getSequenceNumber()
```

Get the sequence number.

Sequence numbers are normally assigned in the LogRecord constructor, which assigns unique sequence numbers to each new LogRecord in increasing order.

Returns:

`the sequence number`

setSequenceNumber

```
public void setSequenceNumber(long seq)
```

Set the sequence number.

Sequence numbers are normally assigned in the LogRecord constructor, so it should not normally be necessary to use this method.

Parameters:

`seq - the sequence number`

getSourceClassName

```
public String getSourceClassName()
```

Get the name of the class that (allegedly) issued the logging request.

Note that this sourceClassName is not verified and may be spoofed. This information may either have been provided as part of the logging call, or it may have been inferred automatically by the logging framework. In the latter case, the information may only be approximate and may in fact describe an earlier call on the stack frame.

May be null if no information could be obtained.

Returns:

`the source class name`

setSourceClassName

```
public void setSourceClassName(String sourceClassName)
```

Set the name of the class that (allegedly) issued the logging request.

Parameters:

`sourceClassName - the source class name (may be null)`

getSourceMethodName

```
public String getSourceMethodName()
```

Get the name of the method that (allegedly) issued the logging request.

Note that this sourceMethodName is not verified and may be spoofed. This information may either have been provided as part of the logging call, or it may have been inferred automatically by the logging framework. In the latter case, the information may only be approximate and may in fact describe an earlier call on the stack frame.

May be null if no information could be obtained.

Returns:

`the source method name`

setSourceMethodName

```
public void setSourceMethodName(String sourceMethodName)
```

Set the name of the method that (allegedly) issued the logging request.

Parameters:

`sourceMethodName - the source method name (may be null)`

getMessage

```
public String getMessage()
```

Get the "raw" log message, before localization or formatting.

May be null, which is equivalent to the empty string "".

This message may be either the final text or a localization key.

During formatting, if the source logger has a localization ResourceBundle and if that ResourceBundle has an entry for this message string, then the message string is replaced with the localized value.

Returns:

`the raw message string`

setMessage

```
public void setMessage(String message)
```

Set the "raw" log message, before localization or formatting.

Parameters:

`message - the raw message string (may be null)`

getParameters

```
public Object[] getParameters()
```

Get the parameters to the log message.

Returns:

`the log message parameters. May be null if there are no parameters.`

setParameters

```
public void setParameters(Object[] parameters)
```

Set the parameters to the log message.

Parameters:

`parameters - the log message parameters. (may be null)`

getThreadID

```
public int getThreadID()
```

Get an identifier for the thread where the message originated.

This is a thread identifier within the Java VM and may or may not map to any operating system ID.

Returns:

`thread ID`

setThreadID

```
public void setThreadID(int threadID)
```

Set an identifier for the thread where the message originated.

Parameters:

`threadID - the thread ID`

getMillis

```
public long getMillis()
```

Get event time in milliseconds since 1970.

Returns:

`event time in millis since 1970`

setMillis

```
public void setMillis(long millis)
```

Set event time.

Parameters:

`millis - event time in millis since 1970`

getThrown

```
public Throwable getThrown()
```

Get any throwable associated with the log record.

If the event involved an exception, this will be the exception object.
Otherwise null.

Returns:

`a throwable`

setThrown

```
public void setThrown(Throwable thrown)
```

Set a throwable associated with the log event.

Parameters:

```
thrown - a throwable (may be null)
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.